

C++ BCA-I(Sem-II) 2013-14

Operator overloading:

- We know that every programming language having rich set of operators and each operator perform its own task.
- But C++ language provides such facility to give our own task to certain operator such that it performs given task instead of its own task.
- Definition: "Forcing particular operator to perform certain task (User defined task) is called as operator overloading"

OR

- "Attaching some additional meaning to existing operator is called as operator overloading"
- *Note that*: while we overload the operator, overloaded operator function executes our task but original work of overloaded operator can't change permanently.

There are some following operators that can't be overloaded:

- 1) Scope resolution operator (: :)
- 2) Conditional (Ternary) operator (? and :)
- 3) Sizeof operator
- 4) Class member access operator (. and .*)

Rules for overloading the operator:

- 1) Only existing operators can be overloaded. New operators cannot be created or overloaded
- 2) We cannot change the basic meaning of an operator.
- 3) Overloaded operators follow the syntax rules of the original operators.
- 4) Overloading arithmetic unary operator by using member function, it take no explicit arguments and no return value.
- 5) Overloading arithmetic unary operator by using friend function, it take one argument i.e. the object of the class and does not return value.
- 6) Overloading arithmetic binary operator by using member function, it take one Argument (Object of class) and also return value.
- 7) Overloading arithmetic binary operator by using friend function, it takes two explicit Arguments and also return value..
- 8) When arithmetic binary operator overloading is done by using member function then the left hand operand must be an object of the class.
- 9) Binary arithmetic operators such as +, -, *, / must explicitly return the value.

Operator overloading is done by two ways:

- I) Operator overloading by using member function.
- II) Operator overloading by using friend function.

1) Overloading operator By using Member function:

Following syntax shows overloading the operator by using member function:

Syntax:

Return_type	operator	symbol(arg1, arg2, , argN);
-------------	----------	--------------------------------------

Here;

'Return_type' is data type that suggests value return from overloaded function.

'operator' is the keyword used to overload the operator.

'Symbol' is the symbol of overloading operator.

E.g.

```
void operator + ( );
```

- There are two kinds of operators viz *unary and binary operator*.
- Also we can overload them by using member function and friend function.

a) Overloading – as unary operator by using member function: Following program shows overloading of – operator : <pre> #include<iostream.h> #include<conio.h> class over { int a; public: void get(); void operator -(); // declaration of operator function void show(); }; void over::get() { cout<<"\n Enter any number "; cin>>a; } void over::operator -() // definition of operator function { a = - a; } void over::show() { cout<<"\n Value = "<a; } void main() { over x; clrscr(); x.get(); -x; // call to operator function x.show(); getch(); } </pre> OUT PUT: Enter any number 45 Value = -45	b) Overloading + as binary operator by using member function: Following program shows overloading of + operator : <pre> #include<iostream.h> #include<conio.h> class over { float a; public: void get(); over operator +(over); void show(); }; void over::get() { cout<<"\n Enter any number "; cin>>a; } over over::operator +(over x) { over t; t.a = a/x.a; return(t); } void over::show() { cout<<"\n Value = "<a; } void main() { over m,n,z; clrscr(); m.get(); n.get(); z=m+n; // call to operator function z.show(); getch(); } </pre> OUT PUT: Enter any number 5 Enter any number 3 Value = 1.666667
--	---

2) Overloading operator By using Friend function:

Following syntax shows overloading the operator by using friend function:
Syntax:

friend Return_type operator symbol(arg1, arg2, , argN);

Here;

- ‘Friend’ is the keyword used to declare the friend function.
 - ‘Return_type’ is data type that suggests value return overloaded function.
 - ‘operator’ is the keyword used to overload the operator.
 - ‘symbol’ is the symbol of overloading operator.
- E.g.

friend void operator + ();

a) Overloading – as unary operator by using friend function: Following program shows overloading of – operator : <pre> #include<iostream.h> #include<conio.h> class over { float a; public: void get(); friend void operator -(over); //declaration of operator fun }; void over::get() { cout<<"\n Enter any number "; cin>>a; } void operator -(over x) // definition operator function { if(x.a%2==0) { cout<<"\n Number is Even"; } else { cout<<"\nNumber is Odd"; } } void main() { over m; clrscr(); m.get(); -m; // call to operator function getch(); } </pre> OUT PUT: Enter any number 65 Number is Odd	b) Overloading + as binary operator by using friend function: Following program shows the overloading of + as binary operator for concatenation of two strings. <pre> #include<iostream.h> #include<string.h> #include<conio.h> class over { char *str; public: void get(); friend over operator +(over,over); void show(); }; void over::get() { cout<<"\n Enter any string "; cin>>str; } over operator + (over x, over y) { over z; z.str=strcat(x.str,y.str); return(z); } void over::show() { cout<<"\n Concatenation= "<<str; } void main() { over m,n,p; clrscr(); m.get(); n.get(); p=m+n; // call to operator function p.show(); getch(); } </pre> OUT PUT: Enter any string Indian Enter any string Cricket Concatenation= IndianCricket
---	---